

Waze Project

The nuts and bolts of machine learning

Your team is close to completing their user churn project. Previously, you completed a project proposal, and used Python to explore and analyze Waze's user data, create data visualizations, and conduct a hypothesis test. Most recently, you built a binomial logistic regression model based on multiple variables.

Leadership appreciates all your hard work. Now, they want your team to build a machine learning model to predict user churn. To get the best results, your team decides to build and test two tree-based models: random forest and XGBoost.

Your work will help leadership make informed business decisions to prevent user churn, improve user retention, and grow Waze's business.

Course 6 End-of-Course Project: Build a machine learning model

In this activity, you will practice using tree-based modeling techniques to predict on a binary target class.

The purpose of this model is to find factors that drive user churn.

The goal of this model is to predict whether or not a Waze user is retained or churned.

This activity has three parts:

Part 1: Ethical considerations

- Consider the ethical implications of the request
- Should the objective of the model be adjusted?

Part 2: Feature engineering

- Perform feature selection, extraction, and transformation to prepare the data for modeling

Part 3: Modeling

- Build the models, evaluate them, and advise on next steps

Follow the instructions and answer the questions below to complete the activity. Then, you will complete an Executive Summary using the questions listed on the PACE Strategy Document.

Be sure to complete this activity before moving on. The next course item will provide you with a completed exemplar to compare to your own work.

Build a machine learning model



PACE stages

Throughout these project notebooks, you'll see references to the problem-solving framework PACE. The following notebook components are labeled with the respective PACE stage: Plan, Analyze, Construct, and Execute.



PACE: Plan

Consider the questions in your PACE Strategy Document to reflect on the Plan stage.

In this stage, consider the following questions:

1. What are you being asked to do?

Predict if a customer will churn or be retained.

2. What are the ethical implications of the model? What are the consequences of your model making errors?

- What is the likely effect of the model when it predicts a false negative (i.e., when the model says a Waze user won't churn, but they actually will)?

Waze will fail to take proactive measures to retain users who are likely to stop using the app. For example, Waze might proactively push an app notification to users, or send a survey to better understand user dissatisfaction.

- What is the likely effect of the model when it predicts a false positive (i.e., when the model says a Waze user will churn, but they actually won't)?

Waze may take proactive measures to retain users who are NOT likely to churn. This may lead to an annoying or negative experience for loyal users of the app.

3. Do the benefits of such a model outweigh the potential problems?

The proactive measures taken by Waze might have unintended effects on users, and these effects might encourage user churn. Follow-up analysis on the effectiveness of the measures is recommended. If the measures are reasonable and effective, then the benefits will most likely outweigh the problems.

4. Would you proceed with the request to build this model? Why or why not?

Yes. There aren't any significant risks for building such a model.

Task 1. Imports and data loading

Import packages and libraries needed to build and evaluate random forest and XGBoost classification models.

```
In [1]: # Import packages for data manipulation
import numpy as np
import pandas as pd

# Import packages for data visualization
import matplotlib.pyplot as plt

# This lets us see all of the columns, preventing Jupyter from redacting them.
pd.set_option('display.max_columns', None)

# Import packages for data modeling
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import roc_auc_score, roc_curve, auc
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
f1_score, confusion_matrix, ConfusionMatrixDisplay, RocCurveDisplay, Precision

from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# This is the function that helps plot feature importance
from xgboost import plot_importance

# This module lets us save our models once we fit them.
import pickle
```

Now read in the dataset as `df0` and inspect the first five rows.

```
In [2]: # Import dataset
df0 = pd.read_csv('waze_dataset.csv')
```

```
In [3]: # Inspect the first five rows
df0.head()
```

```
Out[3]:
```

	ID	label	sessions	drives	total_sessions	n_days_after_onboarding	total_navigations_fav1
0	0	retained	283	226	296.748273	2276	208
1	1	retained	133	107	326.896596	1225	19
2	2	retained	114	95	135.522926	2651	0
3	3	retained	49	40	67.589221	15	322
4	4	retained	84	68	168.247020	1562	166

PACE: Analyze

Consider the questions in your PACE Strategy Document to reflect on the Analyze stage.



Task 2. Feature engineering

You have already prepared much of this data and performed exploratory data analysis (EDA) in previous courses. You know that some features had stronger correlations with churn than others, and you also created some features that may be useful.

In this part of the project, you'll engineer these features and some new features to use for modeling.

To begin, create a copy of `df0` to preserve the original dataframe. Call the copy `df`.

```
In [4]: # Copy the df0 dataframe
df = df0.copy()
```

Call `info()` on the new dataframe so the existing columns can be easily referenced.

```
In [5]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 13 columns):
 #   Column                                Non-Null Count  Dtype  
---  -
 0   ID                                    14999 non-null  int64  
 1   label                                14299 non-null  object  
 2   sessions                             14999 non-null  int64  
 3   drives                               14999 non-null  int64  
 4   total_sessions                       14999 non-null  float64 
 5   n_days_after_onboarding              14999 non-null  int64  
 6   total_navigations_fav1               14999 non-null  int64  
 7   total_navigations_fav2               14999 non-null  int64  
 8   driven_km_drives                     14999 non-null  float64 
 9   duration_minutes_drives               14999 non-null  float64 
10  activity_days                         14999 non-null  int64  
11  driving_days                          14999 non-null  int64  
12  device                               14999 non-null  object  
dtypes: float64(3), int64(8), object(2)
memory usage: 1.5+ MB
```

km_per_driving_day

1. Create a feature representing the mean number of kilometers driven on each driving day in the last month for each user. Add this feature as a column to `df`.
2. Get descriptive statistics for this new feature

```
In [6]: # 1. Create `km_per_driving_day` feature
df['km_per_driving_day'] = df['driven_km_drives'] / df['driving_days']

# 2. Get descriptive stats
df['km_per_driving_day'].describe()
```

```
Out[6]: count      1.499900e+04
mean              inf
std              NaN
min       3.022063e+00
25%       1.672804e+02
50%       3.231459e+02
75%       7.579257e+02
max              inf
Name: km_per_driving_day, dtype: float64
```

Notice that some values are infinite. This is the result of there being values of zero in the `driving_days` column. Pandas imputes a value of infinity in the corresponding rows of the new column because division by zero is undefined.

1. Convert these values from infinity to zero. You can use `np.inf` to refer to a value of infinity.
2. Call `describe()` on the `km_per_driving_day` column to verify that it worked.

```
In [7]: # 1. Convert infinite values to zero
df.loc[df['km_per_driving_day']==np.inf, 'km_per_driving_day'] = 0

# 2. Confirm that it worked
df['km_per_driving_day'].describe()
```

```
Out[7]: count      14999.000000
mean         578.963113
std         1030.094384
min           0.000000
25%         136.238895
50%         272.889272
75%         558.686918
max        15420.234110
Name: km_per_driving_day, dtype: float64
```

percent_sessions_in_last_month

1. Create a new column `percent_sessions_in_last_month` that represents the percentage of each user's total sessions that were logged in their last month of use.
2. Get descriptive statistics for this new feature

```
In [8]: # 1. Create `percent_sessions_in_last_month` feature
df['percent_sessions_in_last_month'] = df['sessions'] / df['total_sessions']

# 2. Get descriptive stats
df['percent_sessions_in_last_month'].describe()
```

```
Out[8]: count      14999.000000
mean         0.449255
std          0.286919
min          0.000000
25%          0.196221
50%          0.423097
75%          0.687216
max          1.530637
Name: percent_sessions_in_last_month, dtype: float64
```

professional_driver

Create a new, binary feature called `professional_driver` that is a 1 for users who had 60 or more drives **and** drove on 15+ days in the last month.

Note: The objective is to create a new feature that separates professional drivers from other drivers. In this scenario, domain knowledge and intuition are used to determine these deciding thresholds, but ultimately they are arbitrary.

To create this column, use the `np.where()` (<https://numpy.org/doc/stable/reference/generated/numpy.where.html>) function. This function accepts as arguments:

1. A condition
2. What to return when the condition is true
3. What to return when the condition is false

Example:

```
x = [1, 2, 3]
x = np.where(x > 2, 100, 0)
x
array([ 0,  0, 100])
```

```
In [9]: # Create `professional_driver` feature
df['professional_driver'] = np.where((df['drives'] >= 60) & (df['driving_days']
```

total_sessions_per_day

Now, create a new column that represents the mean number of sessions per day *since onboarding*.

```
In [10]: # Create `total_sessions_per_day` feature
df['total_sessions_per_day'] = df['total_sessions'] / df['n_days_after_onboard
```

As with other features, get descriptive statistics for this new feature.

```
In [11]: # Get descriptive stats
df['total_sessions_per_day'].describe()
```

```
Out[11]: count      14999.000000
mean         0.338698
std          1.314333
min          0.000298
25%          0.051037
50%          0.100775
75%          0.216269
max          39.763874
Name: total_sessions_per_day, dtype: float64
```

km_per_hour

Create a column representing the mean kilometers per hour driven in the last month.

```
In [12]: # Create `km_per_hour` feature
df['km_per_hour'] = df['driven_km_drives'] / (df['duration_minutes_drives'] /
df['km_per_hour']).describe()
```

```
Out[12]: count      14999.000000
mean         190.394608
std          334.674026
min           72.013095
25%           90.706222
50%          122.382022
75%          193.130119
max          23642.920871
Name: km_per_hour, dtype: float64
```

These numbers are obviously problematic, and it would be worthwhile to seek clarification from Waze regarding how these features are collected to better understand why such unrealistic speeds are observed.

km_per_drive

Create a column representing the mean number of kilometers per drive made in the last month for each user. Then, print descriptive statistics for the feature.

```
In [13]: # Create `km_per_drive` feature
df['km_per_drive'] = df['driven_km_drives'] / df['drives']
df['km_per_drive'].describe()
```

```
Out[13]: count      1.499900e+04
mean              inf
std              NaN
min       1.008775e+00
25%       3.323065e+01
50%       7.488006e+01
75%       1.854667e+02
max              inf
Name: km_per_drive, dtype: float64
```

This feature has infinite values too. Convert the infinite values to zero, then confirm that it

```
In [14]: # 1. Convert infinite values to zero
df.loc[df['km_per_drive']==np.inf, 'km_per_drive'] = 0

# 2. Confirm that it worked
df['km_per_drive'].describe()
```

```
Out[14]: count      14999.000000
mean         232.817946
std          620.622351
min           0.000000
25%          32.424301
50%          72.854343
75%         179.347527
max         15777.426560
Name: km_per_drive, dtype: float64
```

percent_of_sessions_to_favorite

Finally, create a new column that represents the percentage of total sessions that were used to navigate to one of the users' favorite places. Then, print descriptive statistics for the new column.

This is a proxy representation for the percent of overall drives that are to a favorite place. Since total drives since onboarding are not contained in this dataset, total sessions must serve as a reasonable approximation.

People whose drives to non-favorite places make up a higher percentage of their total drives might be less likely to churn, since they're making more drives to less familiar places.

```
In [15]: # Create `percent_of_sessions_to_favorite` feature
df['percent_of_drives_to_favorite'] = (
    df['total_navigations_fav1'] + df['total_navigations_fav2']) / df['total_s

# Get descriptive stats
df['percent_of_drives_to_favorite'].describe()
```

```
Out[15]: count      14999.000000
mean         1.665439
std          8.865666
min           0.000000
25%          0.203471
50%          0.649818
75%          1.638526
max          777.563629
Name: percent_of_drives_to_favorite, dtype: float64
```

Task 3. Drop missing values

Because you know from previous EDA that there is no evidence of a non-random cause of the 700 missing values in the `label1` column, and because these observations comprise less than 5% of the data, use the `dropna()` method to drop the rows that are missing this data.


```
In [16]: # Drop rows with missing values
df = df.dropna(subset=['label'])
```

Task 4. Outliers

You know from previous EDA that many of these columns have outliers. However, tree-based models are resilient to outliers, so there is no need to make any imputations.

Task 5. Variable encoding

Dummying features

In order to use `device` as an X variable, you will need to convert it to binary, since this variable is categorical.

In cases where the data contains many categorical variables, you can use pandas built-in `pd.get_dummies()` (https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html), or you can use scikit-learn's `OneHotEncoder()` (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>) function.

Note: Each possible category of each feature will result in a feature for your model, which could lead to an inadequate ratio of features to observations and/or difficulty understanding your model's predictions.

Because this dataset only has one remaining categorical feature (`device`), it's not necessary to use one of these special functions. You can just implement the transformation directly.

Create a new, binary column called `device2` that encodes user devices as follows:

- Android -> 0
- iPhone -> 1

```
In [17]: # Create new `device2` variable
df['device2'] = np.where(df['device']=='Android', 0, 1)
df[['device', 'device2']].tail()
```

```
Out[17]:
```

	device	device2
14994	iPhone	1
14995	Android	0
14996	iPhone	1
14997	iPhone	1
14998	iPhone	1

Target encoding

The target variable is also categorical, since a user is labeled as either "churned" or "retained." Change the data type of the `label` column to be binary. This change is needed to train the models.

Assign a 0 for all retained users.

Assign a 1 for all churned users.

Save this variable as `label2` so as not to overwrite the original `label` variable.

Note: There are many ways to do this. Consider using `np.where()` as you did earlier in this notebook.

```
In [18]: # Create binary `label2` column
df['label2'] = np.where(df['label']=='churned', 1, 0)
df[['label', 'label2']].tail()
```

```
Out[18]:
```

	label	label2
14994	retained	0
14995	retained	0
14996	retained	0
14997	churned	1
14998	retained	0

Task 6. Feature selection

Tree-based models can handle multicollinearity, so the only feature that can be cut is `ID`, since it doesn't contain any information relevant to churn.

Note, however, that `device` won't be used simply because it's a copy of `device2`.

Drop `ID` from the `df` dataframe.

```
In [19]: # Drop `ID` column
df = df.drop(['ID'], axis=1)
```

Task 7. Evaluation metric

Before modeling, you must decide on an evaluation metric. This will depend on the class balance of the target variable and the use case of the model.

First, examine the class balance of your target variable.

```
In [20]: # Get class balance of 'label' col
df['label'].value_counts(normalize=True)
```

```
Out[20]: label
retained    0.822645
churned     0.177355
Name: proportion, dtype: float64
```

Approximately 18% of the users in this dataset churned. This is an unbalanced dataset, but not extremely so. It can be modeled without any class rebalancing.

Now, consider which evaluation metric is best. Remember, accuracy might not be the best gauge of performance because a model can have high accuracy on an imbalanced dataset and still fail to predict the minority class.

It was already determined that the risks involved in making a false positive prediction are



PACE: Construct

Consider the questions in your PACE Strategy Document to reflect on the Construct stage.

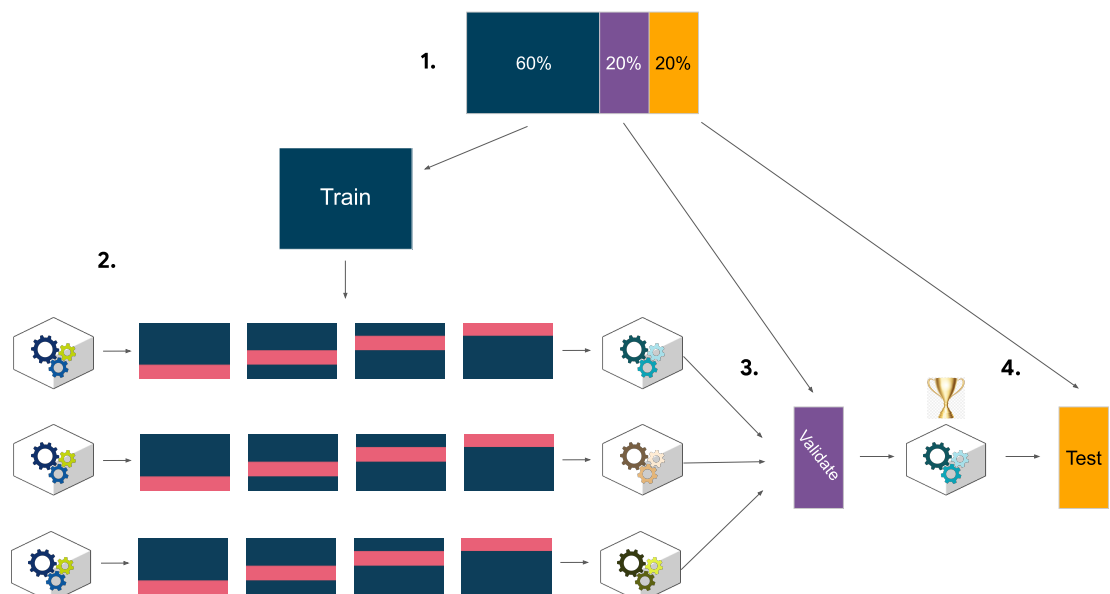
Task 8. Modeling workflow and model selection process

The final modeling dataset contains 14,299 samples. This is towards the lower end of what might be considered sufficient to conduct a robust model selection process, but still doable.

1. Split the data into train/validation/test sets (60/20/20)

Note that, when deciding the split ratio and whether or not to use a validation set to select a champion model, consider both how many samples will be in each data partition, and how many examples of the minority class each would therefore contain. In this case, a 60/20/20 split would result in ~2,860 samples in the validation set and the same number in the test set, of which ~18%—or 515 samples—would represent users who churn.

2. Fit models and tune hyperparameters on the training set
3. Perform final model selection on the validation set
4. Assess the champion model's performance on the test set



Task 9. Split the data

Now you're ready to model. The only remaining step is to split the data into features/target variable and training/validation/test sets.

1. Define a variable `X` that isolates the features. Remember not to use `device` .
2. Define a variable `y` that isolates the target variable (`label2`).
3. Split the data 80/20 into an interim training set and a test set. Don't forget to stratify the splits, and set the random state to 42.
4. Split the interim training set 75/25 into a training set and a validation set, yielding a final ratio of 60/20/20 for training/validation/test sets. Again, don't forget to stratify the splits and set the random state.

```
In [21]: # 1. Isolate X variables
X = df.drop(columns=['label', 'label2', 'device'])

# 2. Isolate y variable
y = df['label2']

# 3. Split into train and test sets
X_tr, X_test, y_tr, y_test = train_test_split(X, y, stratify=y,
                                              test_size=0.2, random_state=42)

# 4. Split into train and validate sets
X_train, X_val, y_train, y_val = train_test_split(X_tr, y_tr, stratify=y_tr,
                                                  test_size=0.25, random_state=42)
```

Verify the number of samples in the partitioned data.

```
In [22]: for x in [X_train, X_val, X_test]:
        print(len(x))
```

```
8579
2860
2860
```

This aligns with expectations.

Task 10. Modeling

Random forest

Begin with using `GridSearchCV` to tune a random forest model.

1. Instantiate the random forest classifier `rf` and set the random state.
2. Create a dictionary `cv_params` of any of the following hyperparameters and their corresponding values to tune. The more you tune, the better your model will fit the data, but the longer it will take.
 - `max_depth`
 - `max_features`
 - `max_samples`
 - `min_samples_leaf`
 - `min_samples_split`
 - `n_estimators`
3. Define a list `scoring` of scoring metrics for `GridSearch` to capture (precision, recall, F1 score, and accuracy).
4. Instantiate the `GridSearchCV` object `rf_cv` . Pass to it as arguments:

- estimator= rf
- param_grid= cv_params
- scoring= scoring
- cv: define the number of cross-validation folds you want (cv=_)
- refit: indicate which evaluation metric you want to use to select the model (refit=_)

refit should be set to 'recall' .

Note: To save time, this exemplar doesn't use multiple values for each parameter in the grid search, but you should include a range of values in your search to home in on the best set of parameters.

```
In [23]: # 1. Instantiate the random forest classifier
rf = RandomForestClassifier(random_state=42)

# 2. Create a dictionary of hyperparameters to tune
cv_params = {'max_depth': [None],
             'max_features': [1.0],
             'max_samples': [1.0],
             'min_samples_leaf': [2],
             'min_samples_split': [2],
             'n_estimators': [300],
             }

# 3. Define a list of scoring metrics to capture
scoring = ['accuracy', 'precision', 'recall', 'f1']

# 4. Instantiate the GridSearchCV object
rf_cv = GridSearchCV(rf, cv_params, scoring=scoring, cv=4, refit='recall')
```

Now fit the model to the training data.

```
In [24]: %%time
rf_cv.fit(X_train, y_train)
```

CPU times: user 2min 37s, sys: 33.4 ms, total: 2min 37s
Wall time: 2min 37s

```
Out[24]: GridSearchCV(cv=4, estimator=RandomForestClassifier(random_state=42),
                    param_grid={'max_depth': [None], 'max_features': [1.0],
                                'max_samples': [1.0], 'min_samples_leaf': [2],
                                'min_samples_split': [2], 'n_estimators': [300]},
                    refit='recall', scoring=['accuracy', 'precision', 'recall', 'f1'])
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Examine the best average score across all the validation folds.

```
In [25]: # Examine best score  
rf_cv.best_score_
```

```
Out[25]: 0.12678201409034398
```

Examine the best combination of hyperparameters.

```
In [26]: # Examine best hyperparameter combo  
rf_cv.best_params_
```

```
Out[26]: {'max_depth': None,  
          'max_features': 1.0,  
          'max_samples': 1.0,  
          'min_samples_leaf': 2,  
          'min_samples_split': 2,  
          'n_estimators': 300}
```

Use the `make_results()` function to output all of the scores of your model. Note that the function accepts three arguments.

HINT

```
In [27]: def make_results(model_name:str, model_object, metric:str):
'''
Arguments:
    model_name (string): what you want the model to be called in the output
    model_object: a fit GridSearchCV object
    metric (string): precision, recall, f1, or accuracy

Returns a pandas df with the F1, recall, precision, and accuracy scores
for the model with the best mean 'metric' score across all validation folds
'''

# Create dictionary that maps input metric to actual metric name in GridSearchCV
metric_dict = {'precision': 'mean_test_precision',
               'recall': 'mean_test_recall',
               'f1': 'mean_test_f1',
               'accuracy': 'mean_test_accuracy',
               }

# Get all the results from the CV and put them in a df
cv_results = pd.DataFrame(model_object.cv_results_)

# Isolate the row of the df with the max(metric) score
best_estimator_results = cv_results.iloc[cv_results[metric_dict[metric]].idxmax()]

# Extract accuracy, precision, recall, and f1 score from that row
f1 = best_estimator_results.mean_test_f1
recall = best_estimator_results.mean_test_recall
precision = best_estimator_results.mean_test_precision
accuracy = best_estimator_results.mean_test_accuracy

# Create table of results
table = pd.DataFrame({'model': [model_name],
                     'precision': [precision],
                     'recall': [recall],
                     'F1': [f1],
                     'accuracy': [accuracy],
                     },
                    )

return table
```

Pass the `GridSearch` object to the `make_results()` function.

```
In [28]: results = make_results('RF cv', rf_cv, 'recall')
results
```

```
Out[28]:
```

	model	precision	recall	F1	accuracy
0	RF cv	0.457163	0.126782	0.198445	0.81851

Aside from the accuracy, the scores aren't that good. However, recall that when you built the logistic regression model in the last course the recall was ~0.09, which means that this model has 33% better recall and about the same accuracy, and it was trained on less data.

If you want, feel free to try retuning your hyperparameters to try to get a better score. You might be able to marginally improve the model.

XGBoost

Try to improve your scores using an XGBoost model.

1. Instantiate the XGBoost classifier `xgb` and set `objective='binary:logistic'`. Also set the random state.
2. Create a dictionary `cv_params` of the following hyperparameters and their corresponding values to tune:
 - `max_depth`
 - `min_child_weight`
 - `learning_rate`
 - `n_estimators`
3. Define a list `scoring` of scoring metrics for grid search to capture (precision, recall, F1 score, and accuracy).
4. Instantiate the `GridSearchCV` object `xgb_cv`. Pass to it as arguments:
 - `estimator= xgb`
 - `param_grid= cv_params`
 - `scoring= scoring`
 - `cv`: define the number of cross-validation folds you want (`cv=_`)
 - `refit`: indicate which evaluation metric you want to use to select the model (`refit='recall'`)

```
In [29]: # 1. Instantiate the XGBoost classifier
xgb = XGBClassifier(objective='binary:logistic', random_state=42)

# 2. Create a dictionary of hyperparameters to tune
cv_params = {'max_depth': [6, 12],
             'min_child_weight': [3, 5],
             'learning_rate': [0.01, 0.1],
             'n_estimators': [300]
            }

# 3. Define a List of scoring metrics to capture
scoring = ['accuracy', 'precision', 'recall', 'f1']

# 4. Instantiate the GridSearchCV object
xgb_cv = GridSearchCV(xgb, cv_params, scoring=scoring, cv=4, refit='recall')
```

Now fit the model to the `X_train` and `y_train` data.

Note this cell might take several minutes to run.


```
In [30]: %%time
xgb_cv.fit(X_train, y_train)
```

CPU times: user 59.6 s, sys: 440 ms, total: 60 s
Wall time: 30.3 s

```
Out[30]: GridSearchCV(cv=4,
                      estimator=XGBClassifier(base_score=None, booster=None,
                                              callbacks=None, colsample_bylevel=None,
                                              colsample_bynode=None,
                                              colsample_bytree=None, device=None,
                                              early_stopping_rounds=None,
                                              enable_categorical=False, eval_metric=N
one,
                                              feature_types=None, gamma=None,
                                              grow_policy=None, importance_type=None,
                                              interaction_constraints=None,
                                              learning_rate=None, ...
                                              max_delta_step=None, max_depth=None,
                                              max_leaves=None, min_child_weight=None,
                                              missing=nan, monotone_constraints=None,
                                              multi_strategy=None, n_estimators=None,
                                              n_jobs=None, num_parallel_tree=None,
                                              random_state=42, ...),
                      param_grid={'learning_rate': [0.01, 0.1], 'max_depth': [6, 12],
                                  'min_child_weight': [3, 5], 'n_estimators': [300]},
                      refit='recall', scoring=['accuracy', 'precision', 'recall', 'f
1'])
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Get the best score from this model.

```
In [31]: # Examine best score
xgb_cv.best_score_
```

```
Out[31]: 0.1708264263019754
```

And the best parameters.

```
In [32]: # Examine best parameters
xgb_cv.best_params_
```

```
Out[32]: {'learning_rate': 0.1,
          'max_depth': 6,
          'min_child_weight': 5,
          'n_estimators': 300}
```

Use the `make_results()` function to output all of the scores of your model. Note that the function accepts three arguments.

```
In [33]: # Call 'make_results()' on the GridSearch object
xgb_cv_results = make_results('XGB cv', xgb_cv, 'recall')
results = pd.concat([results, xgb_cv_results], axis=0)
results
```

```
Out[33]:
```

	model	precision	recall	F1	accuracy
0	RF cv	0.457163	0.126782	0.198445	0.818510
0	XGB cv	0.425932	0.170826	0.243736	0.811866

This model fit the data even better than the random forest model. The recall score is nearly double the recall score from the logistic regression model from the previous course, and it's almost 50% better than the random forest model's recall score, while maintaining a similar accuracy and precision score.

Task 11. Model selection

Now, use the best random forest model and the best XGBoost model to predict on the validation data. Whichever performs better will be selected as the champion model.

Random forest

```
In [34]: # Use random forest model to predict on validation data
rf_val_preds = rf_cv.best_estimator_.predict(X_val)
```

Use the `get_test_scores()` function to generate a table of scores from the predictions on the validation data.

```
In [35]: def get_test_scores(model_name:str, preds, y_test_data):
    """
    Generate a table of test scores.

    In:
        model_name (string): Your choice: how the model will be named in the c
        preds: numpy array of test predictions
        y_test_data: numpy array of y_test data

    Out:
        table: a pandas df of precision, recall, f1, and accuracy scores for y
    """
    accuracy = accuracy_score(y_test_data, preds)
    precision = precision_score(y_test_data, preds)
    recall = recall_score(y_test_data, preds)
    f1 = f1_score(y_test_data, preds)

    table = pd.DataFrame({'model': [model_name],
                          'precision': [precision],
                          'recall': [recall],
                          'F1': [f1],
                          'accuracy': [accuracy]
                          })

    return table
```

```
In [36]: # Get validation scores for RF model
rf_val_scores = get_test_scores('RF val', rf_val_preds, y_val)

# Append to the results table
results = pd.concat([results, rf_val_scores], axis=0)
results
```

```
Out[36]:
```

	model	precision	recall	F1	accuracy
0	RF cv	0.457163	0.126782	0.198445	0.818510
0	XGB cv	0.425932	0.170826	0.243736	0.811866
0	RF val	0.445255	0.120316	0.189441	0.817483

Notice that the scores went down from the training scores across all metrics, but only by very little. This means that the model did not overfit the training data.

XGBoost

Now, do the same thing to get the performance scores of the XGBoost model on the validation data.

```
In [37]: # Use XGBoost model to predict on validation data
xgb_val_preds = xgb_cv.best_estimator_.predict(X_val)

# Get validation scores for XGBoost model
xgb_val_scores = get_test_scores('XGB val', xgb_val_preds, y_val)

# Append to the results table
results = pd.concat([results, xgb_val_scores], axis=0)
results
```

```
Out[37]:
```

	model	precision	recall	F1	accuracy
0	RF cv	0.457163	0.126782	0.198445	0.818510
0	XGB cv	0.425932	0.170826	0.243736	0.811866
0	RF val	0.445255	0.120316	0.189441	0.817483
0	XGB val	0.422680	0.161736	0.233951	0.812238

Just like with the random forest model, the XGBoost model's validation scores were lower, but only very slightly. It is still the clear champion.



PACE: Execute

Consider the questions in your PACE Strategy Document to reflect on the Execute stage.

Task 12. Use champion model to predict on test data

Now, use the champion model to predict on the test dataset. This is to give a final indication of how you should expect the model to perform on new future data, should you decide to use the model.

```
In [38]: # Use XGBoost model to predict on test data
xgb_test_preds = xgb_cv.best_estimator_.predict(X_test)

# Get test scores for XGBoost model
xgb_test_scores = get_test_scores('XGB test', xgb_test_preds, y_test)

# Append to the results table
results = pd.concat([results, xgb_test_scores], axis=0)
results
```

```
Out[38]:
```

	model	precision	recall	F1	accuracy
0	RF cv	0.457163	0.126782	0.198445	0.818510
0	XGB cv	0.425932	0.170826	0.243736	0.811866
0	RF val	0.445255	0.120316	0.189441	0.817483
0	XGB val	0.422680	0.161736	0.233951	0.812238
0	XGB test	0.423963	0.181460	0.254144	0.811189

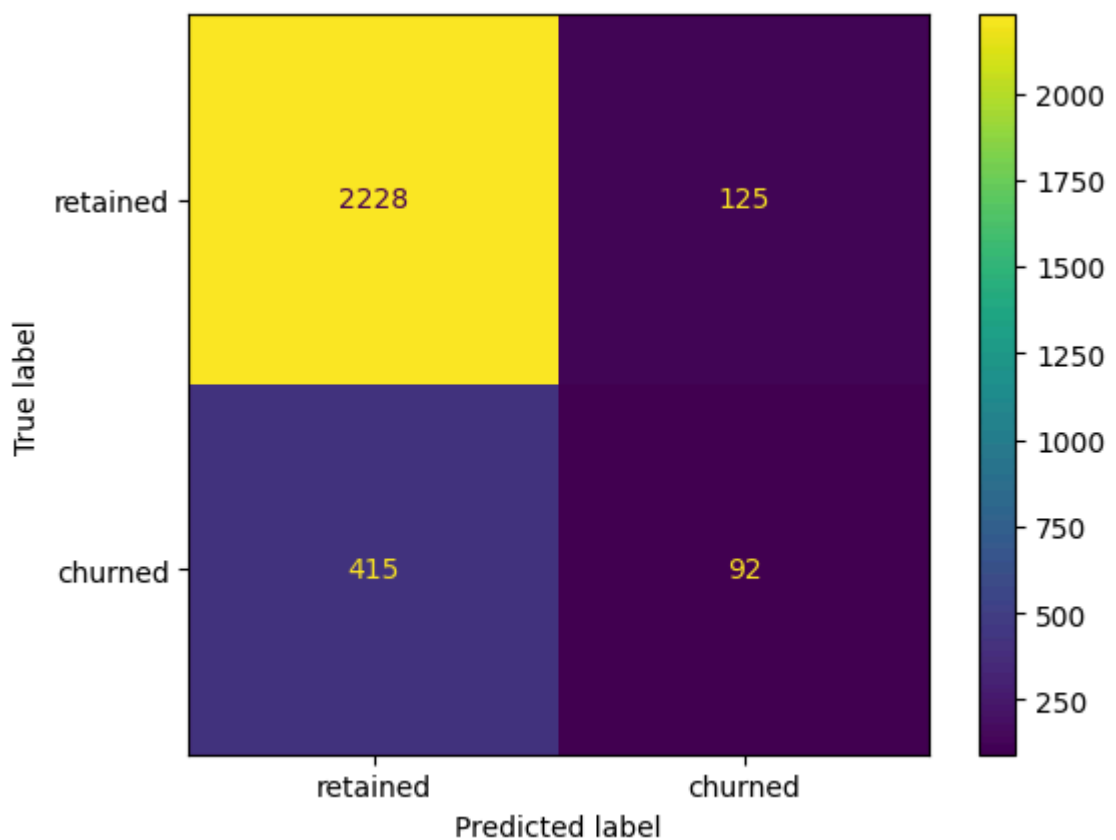
The recall was exactly the same as it was on the validation data, but the precision declined notably, which caused all of the other scores to drop slightly. Nonetheless, this is still within the acceptable range for performance discrepancy between validation and test scores.

Task 13. Confusion matrix

Plot a confusion matrix of the champion model's predictions on the test data.

```
In [39]: # Generate array of values for confusion matrix
cm = confusion_matrix(y_test, xgb_test_preds, labels=xgb_cv.classes_)

# Plot confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['retained', 'churned'])
disp.plot();
```

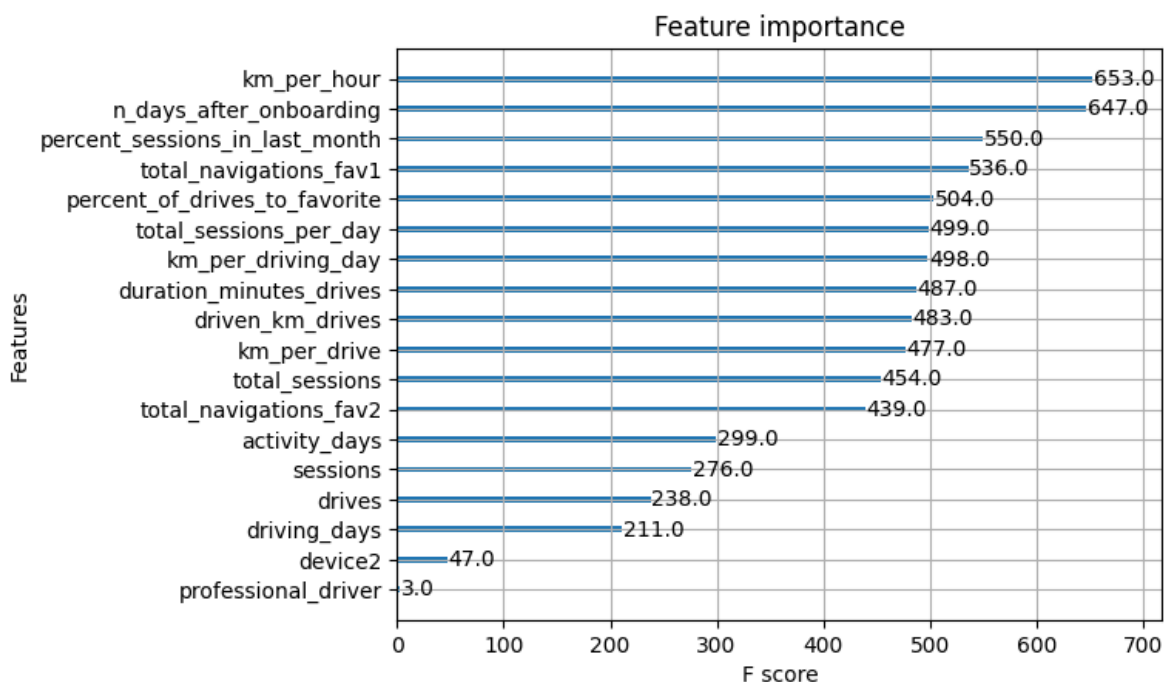


The model predicted over three times as many false negatives than it did false positives, and it correctly identified only 18.1% of the users who actually churned.

Task 14. Feature importance

Use the `plot_importance` function to inspect the most important features of your final model.

```
In [40]: plot_importance(xgb_cv.best_estimator_);
```



The XGBoost model made more use of many of the features than did the logistic regression model from the previous course, which weighted a single feature (`activity_days`) very heavily in its final prediction.

If anything, this underscores the importance of feature engineering. Notice that engineered features accounted for six of the top 10 features (and three of the top five). Feature engineering is often one of the best and easiest ways to boost model performance.

Also, note that the important features in one model might not be the same as the important features in another model. That's why you shouldn't discount features as unimportant without thoroughly examining them and understanding their relationship with the dependent variable, if possible. These discrepancies between features selected by models are typically caused by complex feature interactions.

Remember, sometimes your data simply will not be predictive of your chosen target. This is common. Machine learning is a powerful tool, but it is not magic. If your data does not contain predictive signal, even the most complex algorithm will not be able to deliver consistent and accurate predictions. Do not be afraid to draw this conclusion.

Even if you cannot use the model to make strong predictions, was the work done in vain? What insights can you report back to stakeholders?

Task 15. Conclusion

Now that you've built and tested your machine learning models, the next step is to share your findings with the Waze leadership team. Consider the following questions as you prepare to write your executive summary. Think about key points you may want to share with the team, and what information is most relevant to the user churn project.

Questions:

1. Would you recommend using this model for churn prediction? Why or why not?

It depends. What would the model be used for? If it's used to drive consequential business decisions, then no. The model is not a strong enough predictor, as made clear by its poor recall score. However, if the model is only being used to guide further exploratory efforts, then it can have value.

2. What tradeoff was made by splitting the data into training, validation, and test sets as opposed to just training and test sets?

Splitting the data three ways means that there is less data available to train the model than splitting just two ways. However, performing model selection on a separate validation set enables testing of the champion model by itself on the test set, which gives a better estimate of future performance than splitting the data two ways and selecting a champion model by performance on the test data.

3. What is the benefit of using a logistic regression model over an ensemble of tree-based models (like random forest or XGBoost) for classification tasks?

Logistic regression models are easier to interpret. Because they assign coefficients to predictor variables, they reveal not only which features factored most heavily into their final predictions, but also the directionality of the weight. In other words, they tell you if each feature is positively or negatively correlated with the target in the model's final prediction.

4. What is the benefit of using an ensemble of tree-based models like random forest or XGBoost over a logistic regression model for classification tasks?

Tree-based model ensembles are often better predictors. If the most important thing is the predictive power of the model, then tree-based modeling will usually win out against logistic regression (but not always!). They also require much less data cleaning and require fewer assumptions about the underlying distributions of their predictor variables, so they're easier to work with.

5. What could you do to improve this model?

New features could be engineered to try to generate better predictive signal, as they often do if you have domain knowledge. In the case of this model, the engineered features made up over half of the top 10 most-predictive features used by the model. It could also be helpful to reconstruct the model with different combinations of predictor variables to reduce noise from unproductive features.

6. What additional features would you like to have to help improve the model?

It would be helpful to have drive-level information for each user (such as drive

BONUS

The following content is not required, but demonstrates further steps that you might take to tailor your model to your use case.

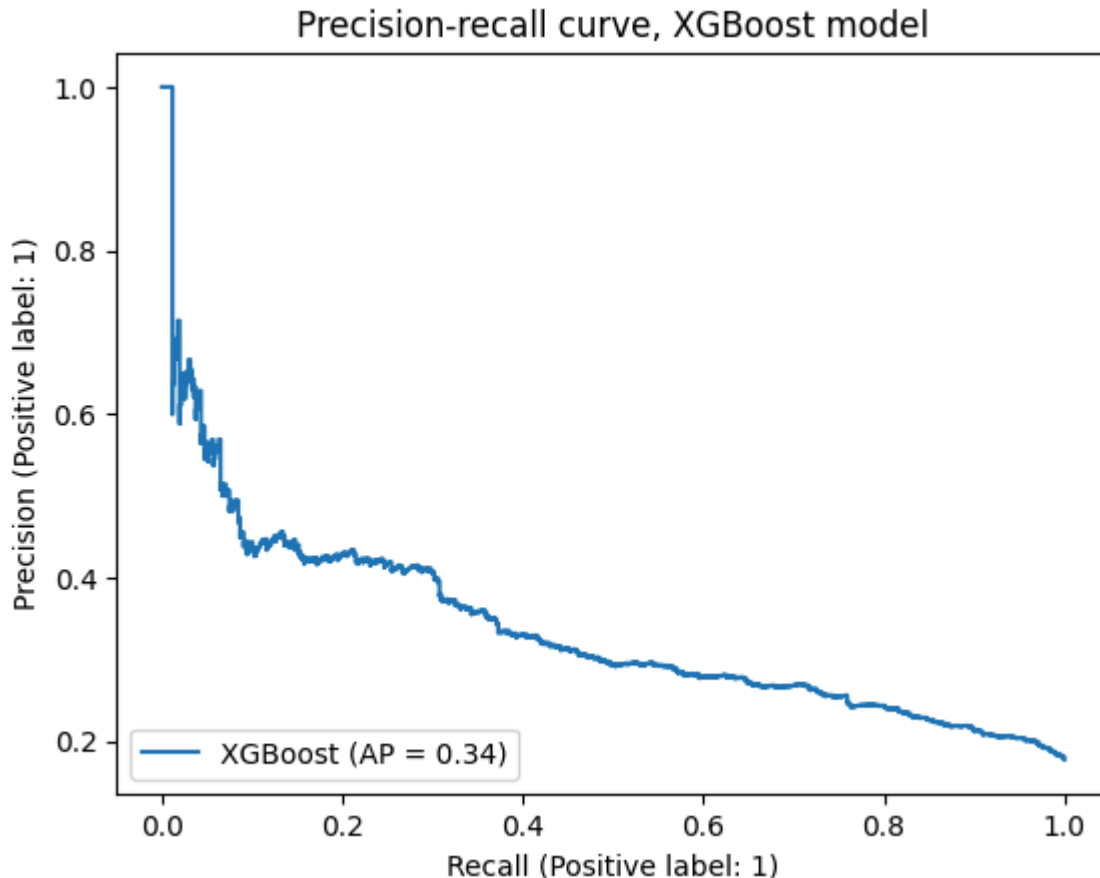
Identify an optimal decision threshold

The default decision threshold for most implementations of classification algorithms—including scikit-learn's—is 0.5. This means that, in the case of the Waze models, if they predicted that a given user had a 50% probability or greater of churning, then that user was assigned a predicted value of 1—the user was predicted to churn.

With imbalanced datasets where the response class is a minority, this threshold might not be ideal. You learned that a precision-recall curve can help to visualize the trade-off between your model's precision and recall.

Here's the precision-recall curve for the XGBoost champion model on the test data.

```
In [41]: # Plot precision-recall curve
display = PrecisionRecallDisplay.from_estimator(
    xgb_cv.best_estimator_, X_test, y_test, name='XGBoost'
)
plt.title('Precision-recall curve, XGBoost model');
```



As recall increases, precision decreases. But what if you determined that false positives aren't much of a problem? For example, in the case of this Waze project, a false positive could just mean that a user who will not actually churn gets an email and a banner notification on their phone. It's very low risk.

So, what if instead of using the default 0.5 decision threshold of the model, you used a lower threshold?

Here's an example where the threshold is set to 0.4:

```
In [42]: # Get predicted probabilities on the test data
predicted_probabilities = xgb_cv.best_estimator_.predict_proba(X_test)
predicted_probabilities
```

```
Out[42]: array([[0.94572806, 0.05427192],
                [0.57626796, 0.42373207],
                [0.98132175, 0.01867823],
                ...,
                [0.8135836 , 0.18641639],
                [0.9661311 , 0.03386891],
                [0.7386279 , 0.26137212]], dtype=float32)
```

The `predict_proba()` method returns a 2-D array of probabilities where each row represents a user. The first number in the row is the probability of belonging to the negative class, the second number in the row is the probability of belonging to the positive class. (Notice that the two numbers in each row are complimentary to each other and sum to one.)

You can generate new predictions based on this array of probabilities by changing the decision threshold for what is considered a positive response. For example, the following code converts the predicted probabilities to {0, 1} predictions with a threshold of 0.4. In other words, any users who have a value ≥ 0.4 in the second column will get assigned a prediction of 1, indicating that they churned.

```
In [43]: # Create a list of just the second column values (probability of target)
probs = [x[1] for x in predicted_probabilities]

# Create an array of new predictions that assigns a 1 to any value >= 0.4
new_preds = np.array([1 if x >= 0.4 else 0 for x in probs])
new_preds
```

```
Out[43]: array([0, 1, 0, ..., 0, 0, 0])
```

```
In [44]: # Get evaluation metrics for when the threshold is 0.4
get_test_scores('XGB, threshold = 0.4', new_preds, y_test)
```

```
Out[44]:
```

	model	precision	recall	F1	accuracy
0	XGB, threshold = 0.4	0.414986	0.284024	0.337237	0.802098

Compare these numbers with the results from earlier.

In [45]: results

Out[45]:

	model	precision	recall	F1	accuracy
0	RF cv	0.457163	0.126782	0.198445	0.818510
0	XGB cv	0.425932	0.170826	0.243736	0.811866
0	RF val	0.445255	0.120316	0.189441	0.817483
0	XGB val	0.422680	0.161736	0.233951	0.812238
0	XGB test	0.423963	0.181460	0.254144	0.811189

Recall and F1 score increased significantly, while precision and accuracy decreased marginally.

So, using the precision-recall curve as a guide, suppose you knew that you'd be satisfied if the model had a recall score of 0.5 and you were willing to accept the ~30% precision score that comes with it. In other words, you'd be happy if the model successfully identified half of the people who will actually churn, even if it means that when the model says someone will churn, it's only correct about 30% of the time.

What threshold will yield this result? There are a number of ways to determine this. Here's one way to do it using a function.

```
In [46]: def threshold_finder(y_test_data, probabilities, desired_recall):
    """
    Find the decision threshold that most closely yields a desired recall score

    Inputs:
        y_test_data: Array of true y values
        probabilities: The results of the `predict_proba()` model method
        desired_recall: The recall that you want the model to have

    Outputs:
        threshold: The decision threshold that most closely yields the desired
        recall: The exact recall score associated with `threshold`
    """
    probs = [x[1] for x in probabilities] # Isolate second column of `probabilities`
    thresholds = np.arange(0, 1, 0.001) # Set a grid of 1,000 thresholds to test

    scores = []
    for threshold in thresholds:
        # Create a new array of {0, 1} predictions based on new threshold
        preds = np.array([1 if x >= threshold else 0 for x in probs])
        # Calculate recall score for that threshold
        recall = recall_score(y_test_data, preds)
        # Append the threshold and its corresponding recall score as a tuple to `scores`
        scores.append((threshold, recall))

    distances = []
    for idx, score in enumerate(scores):
        # Calculate how close each actual score is to the desired score
        distance = abs(score[1] - desired_recall)
        # Append the (index#, distance) tuple to `distances`
        distances.append((idx, distance))

    # Sort `distances` by the second value in each of its tuples (least to greatest)
    sorted_distances = sorted(distances, key=lambda x: x[1], reverse=False)
    # Identify the tuple with the actual recall closest to desired recall
    best = sorted_distances[0]
    # Isolate the index of the threshold with the closest recall score
    best_idx = best[0]
    # Retrieve the threshold and actual recall score closest to desired recall
    threshold, recall = scores[best_idx]

    return threshold, recall
```

Now, test the function to find the threshold that results in a recall score closest to 0.5.

```
In [47]: # Get the predicted probabilities from the champion model
probabilities = xgb_cv.best_estimator_.predict_proba(X_test)

# Call the function
threshold_finder(y_test, probabilities, 0.5)
```

```
Out[47]: (0.194, 0.4990138067061144)
```

Setting a threshold of 0.194 will result in a recall of 0.499.

To verify, you can repeat the steps performed earlier to get the other evaluation metrics for

```
In [49]: # Create an array of new predictions that assigns a 1 to any value >= 0.194
probs = [x[1] for x in probabilities]
new_preds = np.array([1 if x >= 0.194 else 0 for x in probs])

# Get evaluation metrics for when the threshold is 0.194
get_test_scores('XGB, threshold = 0.194', new_preds, y_test)
```

```
Out[49]:
```

	model	precision	recall	F1	accuracy
0	XGB, threshold = 0.194	0.293163	0.499014	0.369343	0.697902

It worked! Hopefully now you understand that changing the decision threshold is another tool that can help you achieve useful results from your model.

Congratulations! You've completed this lab. However, you may not notice a green check mark next to this item on Coursera's platform. Please continue your progress regardless of the check mark. Just click on the "save" icon at the top of this notebook to ensure your work has been logged.